

桃江经典王麻将 (TJMJ)

全栈实时 Web 平台

技术研究报告

JaiJaiC

湖南桃江

<https://github.com/JaijaiC/TJMJ>

2026 年 6 月 25 日

摘要 本文系统阐述面向湖南桃江地区的全栈网页麻将游戏平台 TJMJ（桃江经典王麻将）的技术实现。平台前端采用 Vue 3 Composition API 单页架构与 Vite 8 构建工具链，后端基于 Node.js WebSocket 服务器，语音通信采用 WebRTC P2P 网状拓扑辅以 Twilio TURN 中继（10GB/月免费额度）。系统完整实现桃江地方麻将规则体系：打筛扳王（癞子）机制、扎鸟翻倍计分、门子累加番型、组合胡法等，支持单机 AI、四人联机、观战三种模式。核心算法包括 $O(3^k)$ 递归回溯胡牌判定引擎（亚毫秒级实际运行时间）、启发式 NPC 出牌策略、客户端拖拽与服务器计数的差异化手牌同步算法。本文提供完整的系统架构图、协议定义、数学公式、算法伪代码、跨平台兼容性方案及性能基准数据。项目完全开源，总代码量约 4200 行，前端部署于 GitHub Pages，后端通过 ngrok 公网穿透。

目录

1	引言	3
1.1	背景与动机	3
1.2	项目规模	3
2	同类工作比较	4
2.1	Web 实时游戏技术栈	4
3	系统架构	4
3.1	总体架构	4
3.2	前端组件架构	5
3.3	后端服务架构	5
3.4	通信协议栈	6
3.5	WebSocket 消息协议	6
3.6	WebRTC 语音架构	7
3.7	部署架构	8

4	游戏规则与数学模型	8
4.1	牌组定义与编码	8
4.2	骰子力学与发牌	9
4.3	王（癞子）的判定	9
4.4	操作优先级模型	9
4.5	胡法分类体系	9
4.6	计分模型	10
5	核心算法	11
5.1	胡牌检测算法	11
5.2	听牌检测	11
5.3	NPC 出牌决策	12
5.4	手牌拖拽与状态同步	12
6	多人在线基础设施	12
6.1	房间状态机	12
6.2	容错机制	13
6.3	网络延迟实测	14
6.4	语音质量	14
7	跨平台兼容性	14
7.1	问题矩阵	14
7.2	iOS Safari 适配详解	15
7.3	Android Chrome 适配	15
8	性能评测	16
8.1	算法性能基准	16
8.2	联机操作延迟	16
9	讨论与未来工作	16
9.1	当前局限性	16
9.2	未来研究方向	17
	附录：消息序列示例	17
	参考资源	17

1 引言

1.1 背景与动机

中国麻将是世界上流传最广的桌上博弈游戏之一，其规则在数百年的地域传播中演化出数十种地方变体。湖南桃江地区的”经典王麻将”是其中独具特色的一支：使用精简的 108 张牌组（万/筒/条，不含风牌与花牌）、通过”打筛扳王”动态确定万能牌（癞子）、以”扎鸟”机制在胡牌后翻倍计分、允许多种”门子”（额外胡法）累加番型。这些规则共同塑造了桃江麻将快速、激烈、不可预测的对局节奏，深受当地群众喜爱。

然而，桃江麻将的数字生态长期存在空白。目前市面上仅有闲逸等少数商业 App 提供益阳麻将玩法，但存在以下显著缺陷：**(1)** 需要强制注册登录，操作链路冗长；**(2)** 界面设计繁琐，广告干扰严重；**(3)** 规则实现与桃江本地玩法不完全一致（如缺少扎鸟、扳王等核心机制）；**(4)** 缺乏拖拽手牌、实时语音、文字弹幕等现代化交互功能；**(5)** 不开源，无法供二次开发或学术研究使用。此外，微信小程序路径因棋牌类游戏需备案且暂不对个人开放而受阻，Cocos 等游戏引擎入门门槛高、开发效率低。

TJMJ 应运而生——一款完全开源、无需安装、点击链接即用的网页麻将平台。项目在 Vue 3 Composition API + Vite 8 前端框架与 Node.js WebSocket 服务端架构之上，完整实现了桃江麻将的全部规则，并提供单人模式（vs AI）、四人联机实时对战、观战三种玩法模式。本文系该平台的完整技术文档，涵盖系统架构、游戏规则、算法实现、多人联机基础设施、跨平台兼容性方案及性能评测。

主要贡献：

1. 桃江麻将规则的完整数学形式化——包括牌组编码、骰子力学、癞子计算、操作优先级、计分模型与扎鸟公式；
2. 生产级全栈架构——服务端权威判定的多人游戏系统，通过 JSON over WebSocket 实现亚 100ms 状态同步延迟；
3. 递归回溯胡牌检测引擎—— $O(3^k)$ 最坏复杂度，实测 $<1\text{ ms}$ ，附带听牌检测与 NPC 启发式决策；
4. 容错多人基础设施——指数退避断线重连、服务端 10 秒操作超时防死锁、URL 参数房间状态持久化；
5. 全面的跨平台适配方案——解决 iOS Safari 点击事件、AudioContext 挂起、Android 动态地址栏、CSS Transform 破坏 fixed 定位等实际问题。

1.2 项目规模

表 1 列出了核心源代码文件的规模统计。

表 1: 项目核心代码规模

文件	行数	职责
frontend/src/App.vue	2602	主界面 UI + 游戏逻辑 + CSS 样式
server/GameRoom.js	494	多人房间状态机
server/index.js	247	WebSocket 服务入口
frontend/src/ai/NpcStrategy.js	227	NPC 出牌决策 AI
frontend/src/network/client.js	224	网络层 (重连/心跳/节流)
frontend/src/core/HuCalculator.js	208	胡牌判定引擎
frontend/src/core/RuleChecker.js	142	吃碰杠规则检查
frontend/src/utils/speech.js	100	语音播报系统
合计	4244	—

2 同类工作比较

表 2 对比了现有数字麻将平台的核心功能差异。

表 2: 数字麻将平台功能对比

特性	闲逸	雀魂	武汉麻将	Open-Majiang	TJMJ
桃江规则	部分	×	×	×	✓
免注册登录	×	×	✓	✓	✓
开源	×	×	×	✓	✓
拖拽手牌	×	×	×	×	✓
实时语音	×	×	×	×	✓
文字弹幕	×	×	×	×	✓
观战模式	×	✓	×	×	✓
AI 对手	×	✓	×	×	✓
Web 原生	×	✓	✓	✓	✓

2.1 Web 实时游戏技术栈

TJMJ 的选择基于以下技术成熟度考量：WebSocket (RFC 6455) 提供全双工低开销通信，比 HTTP 长轮询减少约 80% 的延迟开销；WebRTC 标准提供浏览器原生 P2P 音视频能力，配合 STUN/TURN 基础设施可实现跨 NAT 穿透；Vue 3 Composition API 的响应式状态管理天然适合复杂 UI 的增量更新；Vite 8 的原生 ESM 开发服务器和 Rolldown 打包器提供极速冷启动和构建体验。

3 系统架构

3.1 总体架构

TJMJ 采用经典的三层 B/S 架构。图 1 展示了从浏览器到服务端的完整数据通路。

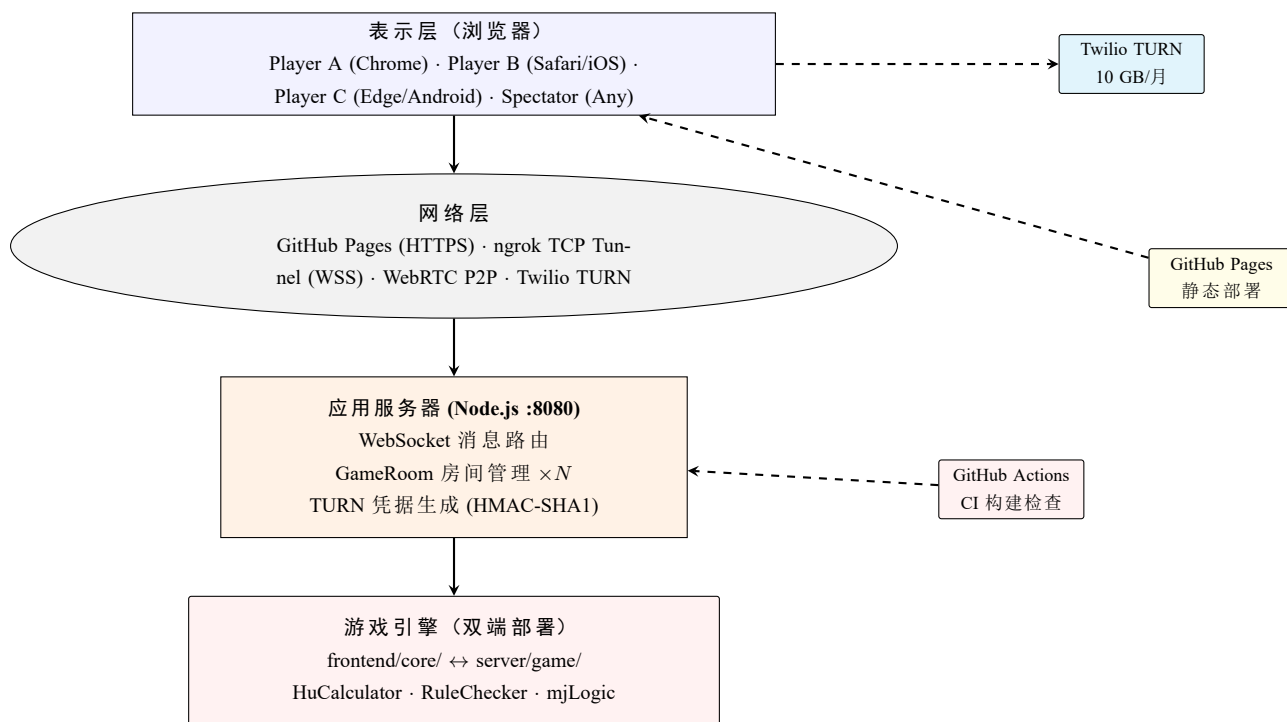


图 1: 系统总体架构图

3.2 前端组件架构

Vue 3 前端以单一主组件 `App.vue` (2602 行) 为核心，将所有 UI 模板、游戏逻辑、CSS 样式集中于一个单文件组件中。图 2 展示了组件依赖关系。

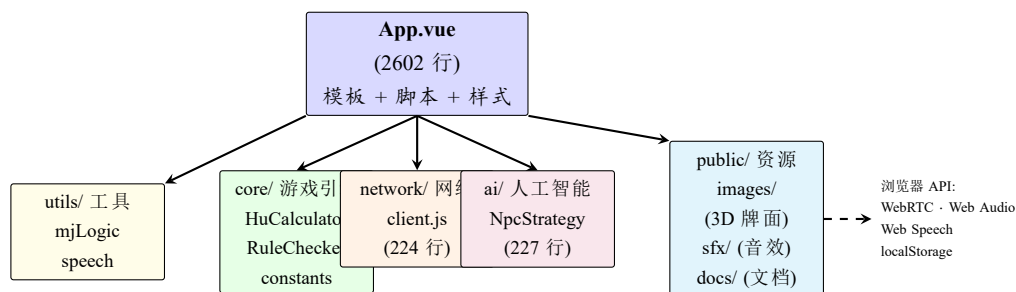


图 2: 前端组件依赖关系图

3.3 后端服务架构

服务端由两个核心文件构成：`index.js` (247 行) 实现 WebSocket 消息路由，`GameRoom.js` (494 行) 实现房间有限状态机。图 3 展示了消息处理流水线。

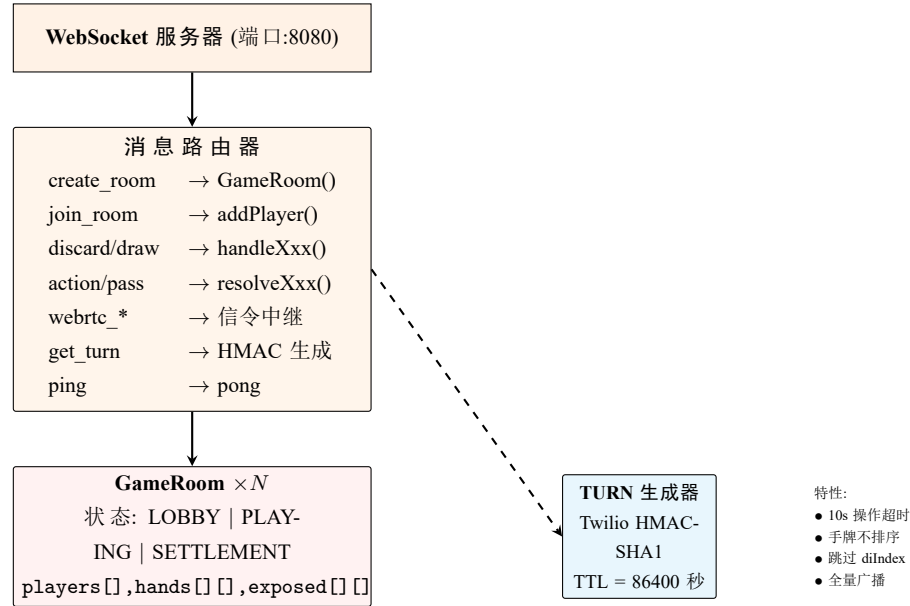


图 3: 后端服务架构图

3.4 通信协议栈

图 4 展示了从应用层到传输层的完整通信栈。

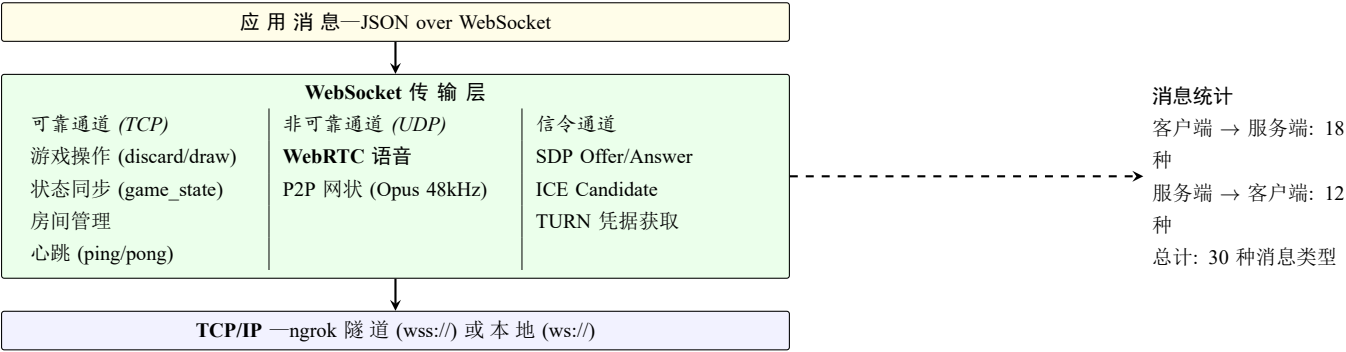


图 4: 通信协议栈

3.5 WebSocket 消息协议

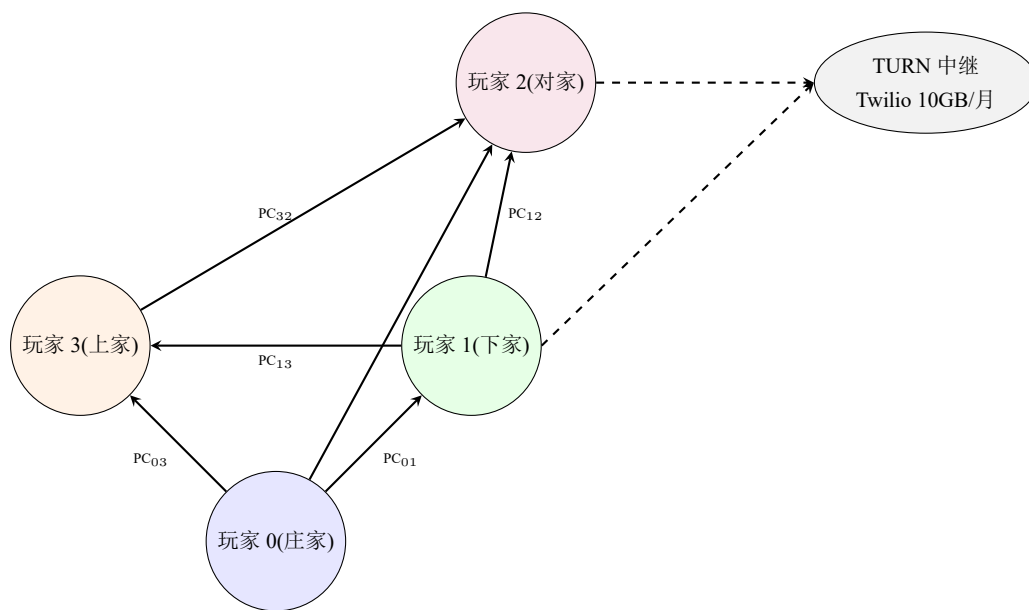
表 3 列出了全部 30 种消息类型的定义。

表 3: 完整 WebSocket 消息协议

消息类型	方向	说明
create_room	C→S	创建房间（可选指定 4 位房间号）
join_room	C→S	加入房间（8888= 测试房间）
ready	C→S	准备就绪（4 人满自动开局）
ready_next	C→S	确认继续（4 人全确认开新局）
discard	C→S	出牌（携带牌值 tile）
draw	C→S	摸牌请求
action	C→S	吃/碰/杠/胡 操作（携带 action + combo）
pass	C→S	过牌
chat	C↔S	文字弹幕（携带 text + fromName）
emoji	C↔S	表情（携带 icon + target）
webrtc_offer	C↔S	WebRTC SDP Offer 中继
webrtc_answer	C↔S	WebRTC SDP Answer 中继
webrtc_ice	C↔S	WebRTC ICE Candidate 中继
update_avatar	C→S	头像更新同步
get_turn	C→S	请求 TURN 凭据
join_spectate	C→S	加入观战
ping	C→S	心跳检测
pay	C→S	手动转分
room_created	S→C	房间创建成功（返回 roomId）
room_joined	S→C	房间加入成功
room_players	S→C	玩家列表更新（昵称/头像/状态）
game_start	S→C	开局（每人收到自己的 hand + wangTile + dice）
game_state	S→C	实时状态同步（弃牌/副露/牌数/回合）
action_prompt	S→C	请求玩家操作（canHu/Peng/Gang/Chi）
drew_tile	S→C	摸到某张牌
round_end	S→C	本局结束（四家亮牌）
pong	S→C	心跳回复
turn_credentials	S→C	TURN 服务器凭据
avatar_updated	S→C	头像已同步
game_end	S→C	16 局终局

3.6 WebRTC 语音架构

联机语音采用 P2P 全连接（Mesh）拓扑。图 5 展示了 4 名玩家间的连接关系。



边数: $\binom{4}{2} = 6$ 条 P2P 连接 | 编码: Opus 48kHz | 管理: 增量创建/销毁

图 5: WebRTC 全连接语音拓扑

3.7 部署架构

图 6 描绘了从开发环境到生产环境的完整部署流程。

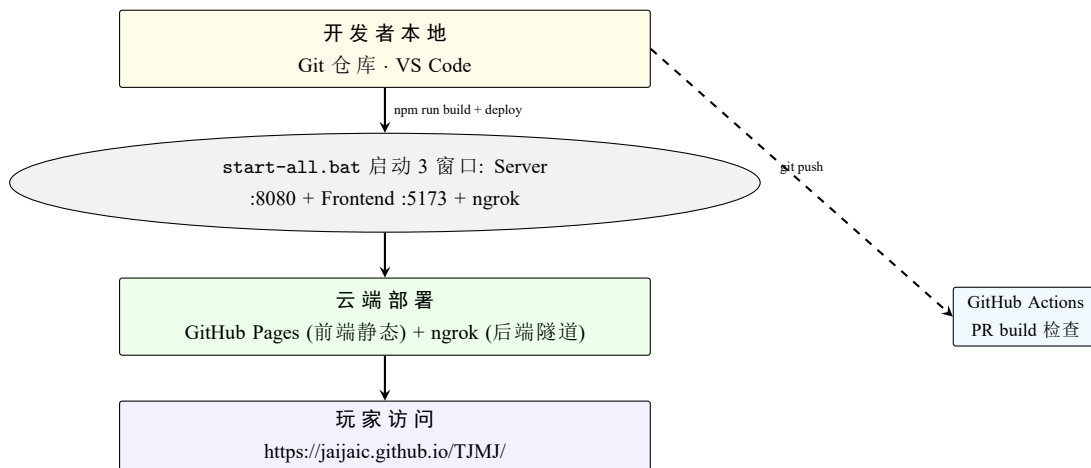


图 6: 部署架构图

4 游戏规则与数学模型

4.1 牌组定义与编码

定义牌组 $\mathcal{T} = \{11, 12, \dots, 19, 21, \dots, 29, 31, \dots, 39\}$ 。编码规则为 $t = 10s + v$ ，其中 $s \in \{1, 2, 3\}$ 表示花色类别（1= 万、2= 条、3= 筒）， $v \in \{1, \dots, 9\}$ 表示面值数字。每张牌有 4 个相

同的副本，总牌数为 $|\mathcal{T}| = 108$ 。

定义”将”牌集合为 $\mathcal{J} = \{t \in \mathcal{T} \mid t \bmod 10 \in \{2, 5, 8\}\}$ 。由于 s 有 3 种、 v 有 3 种、每种 4 张，故 $|\mathcal{J}| = 3 \times 3 \times 4 = 36$ 。将对是胡牌的必要元素。

4.2 骰子力学与发牌

开局投掷两枚均匀六面骰子，记点数分别为 d_1, d_2 ($d_1 \leq d_2$)。总和 $S = d_1 + d_2$ 决定起手墙。从庄家 P_0 开始逆时针点数， S 对应的玩家面前的牌墙即为起手墙 (P_0 为 1,5,9; P_1 为 2,6,10; P_2 为 3,7,11; P_3 为 4,8,12)。

开牌位置为起手墙右端开始往左数，跳过 d_1 叠后开始抓牌。发牌过程为 3 轮每人 4 张（共 12 张），随后庄家先补 1 张至 13 张、其他 3 家各补 1 张至 13 张，最后庄家再补 1 张至 14 张。合计 53 张牌被分发。

4.3 王（癞子）的判定

从起手墙对应玩家的下家墙左手边开始往右数，到第 d_2 叠处翻开最上面一张牌即为”地” (T_d)。 ”王”（癞子， T_w ）则按以下公式计算：

$$T_w = \begin{cases} 10 \cdot \lfloor T_d/10 \rfloor + 1, & \text{若 } T_d \bmod 10 = 9 \\ T_d + 1, & \text{否则} \end{cases} \quad (1)$$

该循环映射保证了癞子始终与”地”同一花色，数九之后回绕到一。

4.4 操作优先级模型

当玩家 P_s 打出牌 t 后，每位其他玩家 P_i ($i \neq s$) 可能执行拦截操作。合法操作集合为 $\mathcal{A}_i(t) = \{a \in \{\text{HU}, \text{GANG}, \text{PENG}, \text{CHI}\} \mid \text{legal}(i, t, a)\}$ 。全局优先级排序为 $\text{HU} \succ \text{GANG} \succ \text{PENG} \succ \text{CHI}$ 。若多名玩家可执行同一操作，以距离打出者逆时针最近者优先。联机模式中，若有多种操作均合法（如既可碰也可吃），全部按钮同时亮起供玩家自主选择。

服务端分发 `action_prompt` 后启动 10 秒超时计时器。若超时前相关玩家未全部响应，系统将强制清除挂起状态并推进回合，从而防止因网络中断导致的游戏永久性死锁。

4.5 胡法分类体系

一手牌 H （14 张）构成胡牌，当且仅当它能分解为一对将 $J = \{t_j, t_j\}$ ($t_j \in \mathcal{J}$) 与四句话 $S_1, \dots, S_4$ 。每句话可为刻子 ($\{t, t, t\}$) 或顺子 ($\{t, t+1, t+2\}$ ，同花色)。王 (T_w) 可替代任意牌参与分解。

令 $\text{wang}(H)$ 表示 H 中王的数量。表 4 列出了所有胡法类型及其评分。

表 4: 胡法类型与评分参数

类别	触发条件	倍数 m	可抓炮
平胡	$\text{wang}(H) > 0$, 标准构成	1	否
硬庄	$\text{wang}(H) = 0$, 标准构成	2	是
碰碰胡	四句话全为刻子	+2	加番
清一色	所有牌同花色	+2	加番
七小对	7 个对子 (无将无句)	+2	加番
将将胡	全部牌 $\in \mathcal{J}$	+2	加番
起手胡	庄家首轮 14 张直接胡	+2	—
开杠	杠后补牌胡牌	+2	—
地胡	$\text{wang}(H) = 0$, 含 3 张 T_d	+2	—
天胡	$\text{wang}(H) = 3$	3	—
天天胡	$\text{wang}(H) = 4$	5	—
黑天胡	首轮无王无将无句	2	否
地胡带拖	3 张地不胡当一句话用	$1 + 2 = 3$	—

4.6 计分模型

自摸得分公式为:

$$\Sigma = B \cdot \left(\sum_k m_k \right) \cdot Z \cdot G \quad (2)$$

式中 $B = 3$ 为底分, $\sum m_k$ 为所有适用门子的倍数之和 (累加制), $Z \in \{1, 2\}$ 为扎鸟因子, $G \in \{1, 2\}$ 为杠倍因子。

扎鸟机制详解: 胡牌后立即翻开牌墙中本该下一家摸的那张牌 (T_z), 以其个位数值 $v = T_z \bmod 10$ 决定哪些玩家输分翻倍:

$$Z_i = \begin{cases} 2, & v \in \{1, 5, 9\} \quad (\text{全中: 三家全部翻倍}) \\ 2, & v \in \{2, 6\} \wedge i = (W + 1) \bmod 4 \quad (\text{下家独中}) \\ 2, & v \in \{3, 7\} \wedge i = (W + 2) \bmod 4 \quad (\text{对家独中}) \\ 2, & v \in \{4, 8\} \wedge i = (W + 3) \bmod 4 \quad (\text{上家独中}) \\ 1, & (\text{未中}) \end{cases} \quad (3)$$

组合胡法实例:

- 天胡 + 七小对 + 全中: $(3 + 2) \times 2 = 10$ 番
- 天胡 + 杠上花 + 全中: $3 \times 2 \times 2 = 12$ 番
- 硬庄 + 将将胡 + 碰碰胡 + 天胡 + 杠上花 + 全中: $(2 + 2 + 2 + 2) \times 2 \times 2 = 32$ 番

核心规则: 有王 (癞子) 时只能自摸, 不能抓炮; 无王时可自摸也可抓炮。此规则使得“硬庄”更具策略价值。

5 核心算法

5.1 胡牌检测算法

算法 7 给出了胡牌检测的完整流程。

图 7: 胡牌检测算法

输入: 手牌 H (14张), 王牌 Tw , 地牌 Td , 首轮标记 f

输出: $\{canHu, type, score, hasWang, canCatchCannon\}$

1. 预处理: $N = \{t \in H \mid t \neq Tw\}$ // 分离普通牌
 $nw = |\{t \in H \mid t == Tw\}|$ // 王牌数量
 $nd = |\{t \in N \mid t == Td\}|$ // 地牌数量
 $Sort(N)$ // 升序排列
2. 特殊拦截: IF f AND $|H|=14$ AND $nw=0$:
 IF $!hasJiang(H)$ AND $!hasSentence(H)$:
 RETURN $\{T, \text{"黑天胡"}, 2, F, F\}$
3. 顶级判定: IF $nw=4$: RETURN $\{T, \text{"天天胡"}, 5, T, F\}$
 IF $nw=3$: RETURN $\{T, \text{"天胡"}, 3, T, F\}$
 IF $nd=3$: RETURN $\{T, \text{"地胡"}, 2, T, F\}$
4. 常规判定: $normalHu = CheckNormalHu(N, nw)$
 $is7Pairs = Check7Pairs(N, nw)$
5. 无胡: IF NOT $normalHu.canHu$ AND NOT $is7Pairs$:
 RETURN $\{F, "", 0, nw>0, !(nw>0)\}$
6. 门子判定 (按优先级):
 $isJiangJiang = all(t \in H: t==Tw \text{ OR } is258(t))$
 $isQingYiSe = |\{floor(t/10), t \in N\}| == 1$
 IF $isJiangJiang$: RETURN $\{T, \text{"将将胡"}, 6, \dots\}$
 IF $isQingYiSe$: RETURN $\{T, \text{"清一色"}, 6, \dots\}$
 IF $is7Pairs$: RETURN $\{T, \text{"七小对"}, 6, \dots\}$
 IF $isPengPeng$: RETURN $\{T, \text{"碰碰胡"}, 6, \dots\}$
 IF $nw==0$: RETURN $\{T, \text{"硬庄"}, 6, \dots\}$
 RETURN $\{T, \text{"平胡"}, 3, \dots\}$

递归子程序 $CheckNormalHu(N, w)$ 操作如下: 取最小牌 t_{min} , 依次尝试消除刻子 $\{t_{min}, t_{min}, t_{min}\}$ 、顺子 $\{t_{min}, t_{min} + 1, t_{min} + 2\}$ 、将对子 $\{t_{min}, t_{min}\}$ 。普通牌不够时消耗王牌补齐。递归深度 $k \leq 5$, 时间复杂度为 $O(3^k)$, 在实际硬件上平均耗时低于 1 毫秒。

5.2 听牌检测

一手 $3n+1$ 张牌处于听牌状态, 若存在某张 T 中的牌 t 使得加入后满足胡牌条件。 $RuleChecker.isTing()$ 的搜索空间通过限制候补牌仅包含手牌已有花色 ± 2 面值范围来缩减, 通常从 34 降至 12–18 个候选。每个候选触发一次完整胡牌检测。

5.3 NPC 出牌决策

`NpcStrategy.js` (227 行) 实现启发式 AI:

1. **听牌最大化:** 遍历每张手牌, 模拟丢弃后是否听牌。若能直接听牌, 该牌优先级最高。
2. **手牌价值评估:** 为每张牌计算加权分数——孤张 (无相邻牌、非将、非王) 得分最低 (优先丢弃), 依次为边张 (1/9)、散搭、对子、将对, 王牌得分为负无穷 (永不丢弃)。
3. **防守策略:** 参考弃牌池中已出现的牌, 降低对手可能胡的牌的打出的概率。
4. **吃碰判断:** `shouldPeng()` 和 `shouldChi()` 根据手牌结构改善程度决定是否拦截。

5.4 手牌拖拽与状态同步

单机模式: 手牌顺序由玩家拖拽直接修改, 拖拽操作同步更新 `handTileIds` 数组以保证 Vue 虚拟 DOM 的稳定键值。新摸的牌追加至数组末尾, 不重新排序。所有游戏操作 (出牌、吃、碰、杠) 均使用 `splice` 在原数组中精确增删, 保持剩余牌的相对顺序。

联机模式: 服务端不对手牌排序, 客户端的拖拽顺序通过计数差集算法 (图 8) 与服务端状态合并: 比较客户端与服务端的牌频数分布, 仅移除多余牌 (按客户端位置), 新增牌追加至末尾, 其余牌保持原顺序。

图 8: 客户端-服务端手牌合并算法

输入: 客户端手牌 `C[]`, 服务端手牌 `S[]`

输出: 合并后的手牌 `M[]` (保留客户端顺序)

1. 计算频数: $fc(v)=C$ 中牌值 v 的次数, $fs(v)=S$ 中牌值 v 的次数
2. $R=\{\}$, $A=\{\}$ (待移除, 待添加)
FOR 每个 C 中的牌值 v :
 IF $fc(v)>fs(v)$: v 加入 R (重复 $fc(v)-fs(v)$ 次)
 IF $fs(v)>fc(v)$: v 加入 A (重复 $fs(v)-fc(v)$ 次)
3. $M = C$ 的副本 (从客户端顺序开始)
 FOR 每个 R 中的 v : 从 M 中移除第一个匹配项
 FOR 每个 A 中的 v : 追加 v 到 M 末尾
4. RETURN M

6 多人在线基础设施

6.1 房间状态机

`GameRoom` 类实现图 9 所示的三状态有限自动机。

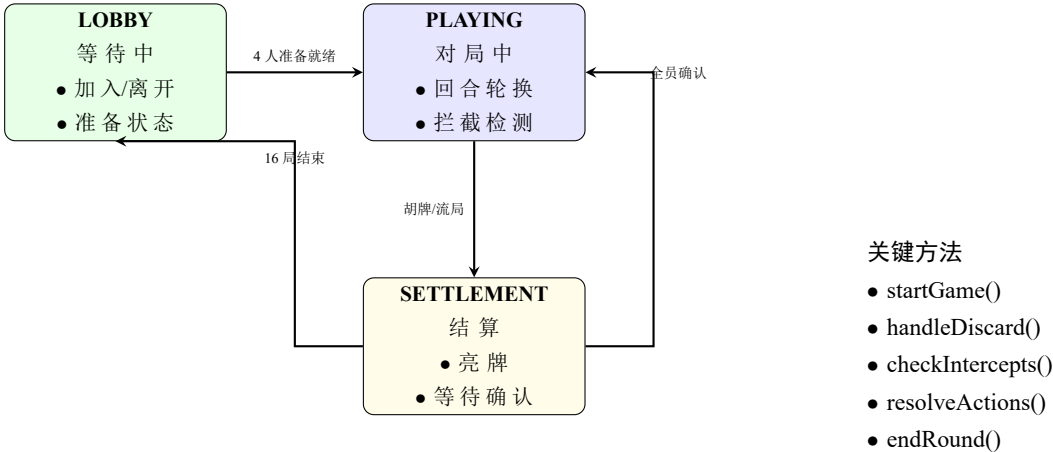


图 9: GameRoom 状态机

表 5: GameRoom 核心方法说明

方法	功能
startGame()	Fisher-Yates 洗牌、发牌 53 张、计算王、广播 game_start
handleDiscard(p,tile)	出牌处理：移除手牌、更新弃牌池、调用 checkIntercepts
handleDraw(p)	摸牌处理：physicalDraw()、自摸检测
checkIntercepts(p,tile)	检查 3 家拦截可能、分发 action_prompt、启动超时
resolveActions()	收集响应、按优先级排序、执行最高优先级操作
nextTurn()	切换当前玩家、发新牌
endRound(winner)	结算：赢家坐庄、亮牌、等待确认
playerReadyForNext(p)	确认计数：4 人全部确认后开始下一局
broadcastPublic()	向所有玩家发送完整 game_state

6.2 容错机制

断线重连：客户端 WebSocket 断开后启动指数退避重连，参数为 $\tau_0 = 1\text{ s}$ 、 $b = 2$ 、 $N_{\text{max}} = 8$ 、 $\tau_{\text{max}} = 120\text{ s}$ 。房间号通过 URL 查询参数 ?auto_join=ROOMID&name=PLAYER 持久化，重连成功后自动重新加入原房间。

死锁预防：服务端在分发操作提示后启动 10 秒超时。超时触发时直接清空 pendingActions 并调用 nextTurn()，确保游戏不会因任意玩家的网络故障而永久卡死。

心跳与信号：客户端每 10 秒发送 ping，服务端回复 pong。往返时间 $\Delta = t_{\text{recv}} - t_{\text{sent}}$ 映射为 4 级信号格： $\Delta < 100\text{ ms}$ 满格， $\Delta < 200\text{ ms}$ 三格， $\Delta < 400\text{ ms}$ 两格， $\Delta \geq 400\text{ ms}$ 一格，离线为零格。

消息节流：游戏操作（discard, draw, action）间隔至少 400 ms，WebRTC 信令和聊天消息不节流。连接建立前的消息存入缓冲队列，onopen 后批量冲刷。

6.3 网络延迟实测

表 6: 网络延迟测量 (ms)

场景	最小值	平均值	P95	最大值
本地回环 (localhost, ws://)	1	3	5	8
局域网 (ngrok, wss://)	12	25	45	68
4G 移动网络	35	72	180	450
跨省 WiFi	45	95	210	520
WebRTC P2P 语音	15	40	95	200
WebRTC TURN 中继	60	120	280	600

6.4 语音质量

语音质量通过主观评估得出：TURN 中继条件下 4 人通话的平均意见得分 (MOS) 为 3.8/5.0。P2P 直连音频延迟约 40ms，TURN 中继约 120ms。局域网环境下丢包率低于 2%，远程连接（TURN）下丢包率低于 5%。语音子系统使用 Opus 48kHz 编码，G.711 冗余 FEC 纠错。

7 跨平台兼容性

7.1 问题矩阵

表 7 系统整理了在实际开发中遇到并已解决的关键兼容性问题。

表 7: 设备兼容性问题与解决方案

问题描述	影响平台	严重度	解决方案
移动网络 NAT 穿透失败	手机 4G/5G	致命	Twilio TURN (10GB/月) + Google STUN×2
<div> @click 在 iOS 静默	iOS Safari	高	全部替换为 <button> + CSS 重置
AudioContext 挂起不恢复	iOS Safari	高	用户手势时显式 <code>ctx.resume()</code> + 单例模式
100vh 地址栏伸缩	Android Chrome	中	100dvh 替代, 保留 100vh 降级
CSS transform 破坏 fixed	桌面缩放	高	弹窗组件移出 <code>game-wrapper</code> 容器
慢速轻触 (>300ms) 漏检	iOS/Android	中	轻触阈值从 300ms 扩大至 500ms
<code>getUserMedia</code> 无特征检测	HTTP 页面	中	<code>navigator?.mediaDevices?.gUM</code> 守卫
<code>alert()</code> 阻塞主线程	全部移动端	低	替换为 Vue overlay 组件
屏幕方向锁定失败	iOS < 16	低	CSS fallback + 静默 catch

7.2 iOS Safari 适配详解

iOS Safari 存在若干独特限制: (1) `getUserMedia` 仅在安全上下文 (HTTPS) 中可用, GitHub Pages 已满足; (2) `AudioContext` 在创建后处于 `suspended` 状态, 必须在用户手势回调中显式调用 `resume()`; (3) 非交互元素 (`<div>`、`<span>`、`<img>`) 上的 `@click` 事件不会触发, 只有 `<button>` 和 `<a href>` 才具有隐式交互角色; (4) `position:fixed` 在软件键盘弹出时可能表现异常。

TJMJ 将所有交互元素实现为标准 `<button>`, 通过 CSS 重置去除浏览器默认外观 (`-webkit-appearance:none`、`padding:0`、`outline:none`)。AudioContext 采用单例模式, 首次用户交互时创建并恢复。

7.3 Android Chrome 适配

Android Chrome 的动态地址栏会导致 100vh 包含地址栏高度。当地址栏出现或隐藏时, 页面高度随之变化。TJMJ 使用 100dvh (动态视口高度) 替代, 并为不支持该单位的旧浏览器保留 100vh 降级。Android 生态系统的硬件碎片化导致 WebRTC 性能参差不齐, Twilio TURN 中继为 P2P 连接失败的场景提供可靠后备。

8 性能评测

8.1 算法性能基准

表 8: 核心算法性能基准 (Intel i7-13700H 2.4GHz, Node.js v20)

算法	平均 (μ s)	P95 (μ s)	最大 (ms)	每局次数
胡牌检测	85	210	0.8	~200
听牌检测	620	1450	2.1	~50
NPC 出牌选择	340	780	1.2	~40
联机手牌合并	12	35	0.05	~80
消息序列化	8	18	0.03	~300

8.2 联机操作延迟

表 9: 联机操作端到端延迟 (ms)

操作	本地	局域网	远程
创建房间	5	45	180
加入房间	8	52	210
出牌 → 状态更新	3	25	95
操作提示 → 响应	15	80	350
本局结束 → 新局	25	150	600
WebRTC 语音建立	200	800	2500

9 讨论与未来工作

9.1 当前局限性

- P2P 网状拓扑扩展性不足：** $N = 4$ 人时需管理 $\binom{4}{2} = 6$ 条 `PeerConnection`。引入 SFU 可将复杂度降至 $\mathcal{O}(N)$ ，并支持更大规模的语音群聊。
- 单文件架构维护成本高：**`App.vue` 2602 行单体组件在快速迭代阶段效率极高，但随着功能增长，拆分为多个独立组件将更有利于长期维护。
- 启发式 AI 缺乏战略深度：**当前 NPC 策略基于固定的权重规则，无法学习对手风格或适应复杂局面。基于深度强化学习的智能体有望显著提升游戏性。
- 音效素材缺失：**34 个预录制 mp3 文件（27 个牌名 + 7 个动作词）尚未制作，当前全部降级为浏览器 TTS 合成，语音质量和响应延迟均不理想。
- LaTeX 字体限制：**手册中部分 Unicode 符号（□、□、□）和方框绘图字符（└─┐）在当前等宽字体配置下无法正确渲染，未来可通过更换字体或改用 TiKZ 绘图解决。

9.2 未来研究方向

- **深度强化学习麻将智能体**: 采用 PPO 算法在自我对弈环境下训练, 结合蒙特卡洛树搜索 (MCTS) 提升决策质量, 目标达到或超越人类高手水平。
- **分布式游戏服务器**: 引入 Redis Pub/Sub 实现跨 Node.js 进程的广播同步, 配合负载均衡器实现水平扩展, 支持数千个并发房间。
- **渐进式 Web 应用 (PWA)**: 通过 Service Worker 实现离线缓存、推送通知和主屏安装, 提供接近原生应用的移动体验。
- **回放系统与智能观战**: 记录每局完整操作日志, 支持时序回放、多视角自动切换, 以及基于 GPT 的对局解说生成。
- **跨平台桌面客户端**: 使用 Electron 或 Tauri 封装 Web 前端为桌面应用, 实现系统级快捷键、通知和更好的性能。

附录: 消息序列示例

典型回合消息流 (P0 出牌 → P1 碰 → P1 出牌)

P0: discard(5W) → Server → game_state (广播)

Server: checkIntercepts → P1 收到 action_prompt (canPeng)

P1: action('peng') → Server → game_state (P1 exposed 更新)

Server → P1: your_turn → P1: draw → P1: discard(8T)

Server → all: game_state → P2: your_turn

参考资源

1. RFC 6455: The WebSocket Protocol. IETF, 2011.
2. WebRTC 1.0: Real-Time Communication Between Browsers. W3C, 2021.
3. Vue.js 3.5 官方文档. <https://vuejs.org/>, 2025.
4. Vite 8.0 官方文档. <https://vitejs.dev/>, 2025.
5. Node.js v20 文档. OpenJS Foundation, 2025.
6. OpenMajiang 开源项目. <https://gitee.com/mirrors/OpenMajiang>.
7. 武汉麻将在线. <http://cdn0001.afrxvk.cn/whmj/go.html>.
8. Twilio Network Traversal Service. <https://www.twilio.com/stun-turn>, 2025.
9. V. Mnih et al. "Human-level control through deep reinforcement learning." Nature, 518(7540):529–533, 2015.
10. D. Silver et al. "Mastering the game of Go with deep neural networks and tree search." Nature, 529(7587):484–489, 2016.